

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



CI/CD & TEST-HARNESS ENGINEERING

Môn học: Kiểm thử phần mềm
Chương trình: Chất lượng cao
Giảng viên: Thầy Trương Phước Lộc

Nhóm: 06
Sinh viên: Nguyễn Ngọc Minh Châu
MSSV: 23127031

Thành phố Hồ Chí Minh, Ngày 3 tháng 7 năm 2026

Mục lục

1	CI/CD Engineering	1
1.1	Khái niệm CI/CD	1
1.2	Quy trình làm việc điển hình theo mô hình CI/CD	1
1.3	Ưu điểm và nhược điểm của CI/CD	2
1.4	Khi nào nên dùng và khi nào không nên dùng quy trình CI/CD	3
1.5	Các công cụ CI/CD phổ biến	3
1.5.1	GitHub Actions	3
1.5.2	Jenkins	4
1.5.3	JetBrains TeamCity	4
1.5.4	CircleCI	4
1.5.5	Azure DevOps	4
1.5.6	GitLab CI	4
1.5.7	Travis CI	4
1.5.8	AWS CodePipeline	5
1.5.9	Bitbucket Pipelines	5
1.6	Các yếu tố so sánh và lựa chọn công cụ CI/CD	5
2	Test Harness Engineering	6
2.1	Khái niệm Test Harness	6
2.2	Cấu tạo của Test Harness	6
2.3	Đặc điểm của Test Harness	7
2.4	Mục đích sử dụng	7
2.5	Quy trình xây dựng Test Harness	7

2.6	Các tình huống áp dụng	8
2.7	Công cụ hỗ trợ Test Harness	8
2.8	Ưu điểm và nhược điểm của Test Harness	8
Tài liệu tham khảo		9

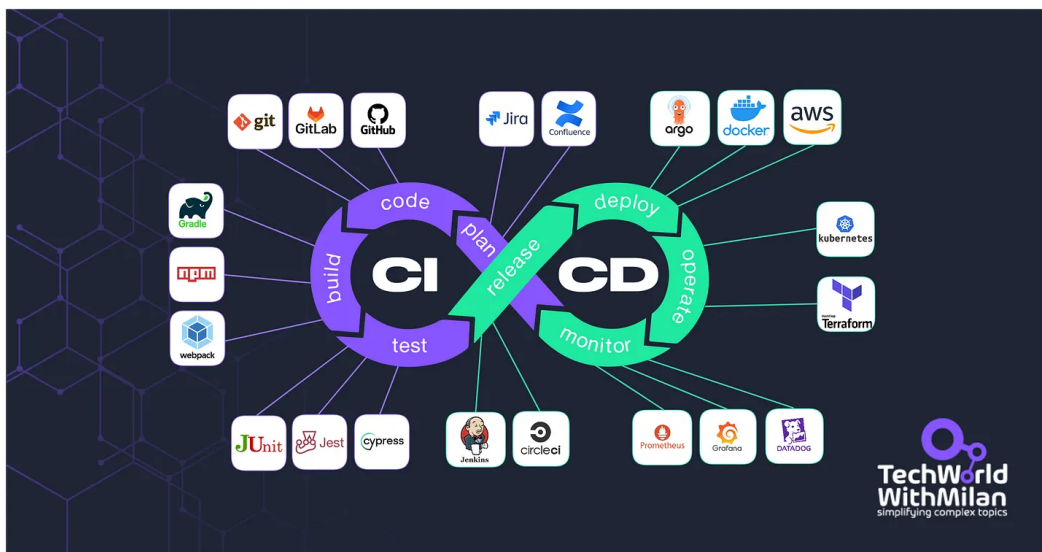
Danh sách hình vẽ

1	Tổng quan về CI/CD	1
2	Quy trình làm việc điển hình của CI/CD pipeline	2
3	Một số công cụ CI/CD phổ biến	5

1 CI/CD Engineering

1.1 Khái niệm CI/CD

Theo ITviec, CI là viết tắt của Continuous Integration (tích hợp liên tục), còn CD có thể là Continuous Delivery (chuyển giao liên tục) hoặc Continuous Deployment (triển khai liên tục). Khái niệm CI/CD thường đề cập đến việc tự động hóa trong quy trình phát triển phần mềm và chuyển giao sản phẩm, giúp việc tích hợp mã nguồn diễn ra nhanh hơn và sản phẩm hoàn thiện được chuyển đến người dùng trong thời gian ngắn nhất.



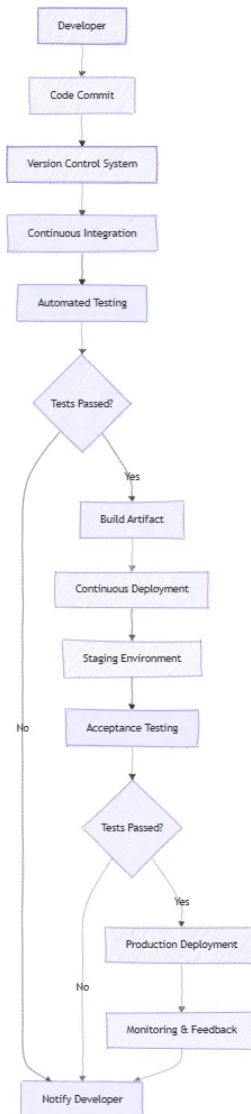
Hình 1: Tổng quan về CI/CD

1.2 Quy trình làm việc điển hình theo mô hình CI/CD

Một pipeline CI/CD thường gồm bốn giai đoạn cơ bản:

1. **Source stage:** Mã nguồn được lấy từ hệ thống quản lý phiên bản như Git, GitHub, GitLab hoặc Bitbucket.
2. **Build stage:** Ứng dụng được biên dịch hoặc build từ source code để tạo ra artifact có thể kiểm thử hoặc triển khai.
3. **Test stage:** Các kiểm thử tự động như unit test, integration test hoặc kiểm thử chất lượng mã được chạy để kiểm tra tính đúng đắn của code.
4. **Deploy stage:** Ứng dụng được triển khai lên môi trường staging hoặc production tùy theo quy trình phát hành của dự án.

Ví dụ thực tế: developer push code lên GitHub, GitHub Actions kích hoạt workflow để build ứng dụng, chạy unit test và integration test. Nếu toàn bộ kiểm thử đều pass, hệ thống sẽ deploy lên staging. Sau đó, một thành viên trong nhóm review và phê duyệt trước khi deploy lên production.



Hình 2: Quy trình làm việc điển hình của CI/CD pipeline

1.3 Ưu điểm và nhược điểm của CI/CD

Ưu điểm:

- Tránh các lỗi không đáng có như lỗi compile hoặc lỗi do môi trường build khác nhau giữa các máy.

- Đảm bảo logic của hệ thống, giúp tính năng mới không phá vỡ tính năng cũ nhờ automation test.
- Giúp developer tập trung vào viết code thay vì phải tự build hoặc deploy thủ công.
- Nâng cao chất lượng code thông qua các ràng buộc trong quy trình phát triển, ví dụ giới hạn quy mô pull request hoặc yêu cầu pipeline pass trước khi merge.
- Phát triển kỹ năng viết unit test cho developer thông qua các chỉ số như code coverage.
- Tối ưu tốc độ phát triển sản phẩm nhờ khả năng theo dõi trạng thái build pipeline và phát hiện lỗi sớm.

Nhược điểm:

- Khi có nhiều pull request cùng lúc, nhóm có thể phải chờ merge, cập nhật lại code khi xảy ra conflict và chạy test lại từ đầu, gây gián đoạn tiến độ.
- Dự án có thể phụ thuộc vào dịch vụ CI/CD bên thứ ba. Nếu dịch vụ gặp sự cố hoặc ngừng hoạt động, quy trình phát triển và triển khai có thể bị ảnh hưởng nghiêm trọng.

1.4 Khi nào nên dùng và khi nào không nên dùng quy trình CI/CD

Nên áp dụng CI/CD càng sớm càng tốt, kể cả với dự án cá nhân, vì hiện nay có nhiều dịch vụ miễn phí hoặc gói miễn phí đủ dùng cho các dự án nhỏ. Một quy trình CI/CD tốt giúp chuẩn hóa cách build, test, deploy và từ đó tối ưu chất lượng công việc.

Tuy nhiên, cần cân nhắc chưa áp dụng hoặc chỉ áp dụng ở mức đơn giản khi tổ chức không có người đủ khả năng vận hành CI/CD, developer chưa nắm rõ công cụ, hoặc nhóm chưa biết cách đảm bảo quy trình hoạt động ổn định. Nếu pipeline gặp sự cố mà nhóm không đủ chuyên môn xử lý, CI/CD có thể gây gián đoạn và làm mất nhiều thời gian hơn so với lợi ích mang lại.

1.5 Các công cụ CI/CD phổ biến

1.5.1 GitHub Actions

GitHub Actions tích hợp chặt chẽ với GitHub và GitHub Enterprise, phù hợp với doanh nghiệp hoặc nhóm phát triển đã sử dụng GitHub làm hệ thống quản lý phiên bản chính. Công cụ này cho phép định nghĩa workflow bằng YAML và tự động hóa các bước build, test, release hoặc deploy ngay trong repository.

1.5.2 Jenkins

Jenkins là công cụ CI/CD mã nguồn mở, chạy trên nền tảng Java, có độ linh hoạt cao và hỗ trợ nhiều kiểu cấu hình thông qua hệ sinh thái plugin phong phú. Tuy nhiên, Jenkins thường cần nhiều thời gian thiết lập và bảo trì hơn, nên phù hợp với tổ chức muốn tự vận hành hạ tầng CI/CD vì yêu cầu bảo mật, pháp lý hoặc kiểm soát nội bộ.

1.5.3 JetBrains TeamCity

JetBrains TeamCity hỗ trợ viết cấu hình CI/CD bằng Kotlin, giúp pipeline có thể được quản lý như mã nguồn. Công cụ này hỗ trợ tốt các hệ sinh thái như Java, .NET, Python, Ruby, Xcode và có thể mở rộng thông qua plugin, phù hợp với nhóm cần khả năng cấu hình rõ ràng và tích hợp đa nền tảng.

1.5.4 CircleCI

CircleCI có ưu điểm là dễ sử dụng, hỗ trợ cả cloud và on-premise, đồng thời tích hợp tốt với GitHub và Bitbucket. Nền tảng này phù hợp với nhóm muốn thiết lập pipeline nhanh, dễ theo dõi và có mô hình giá đơn giản hơn so với các nền tảng tính phí chủ yếu theo số phút build.

1.5.5 Azure DevOps

Azure DevOps cung cấp một bộ công cụ đầy đủ cho quy trình phát triển phần mềm, bao gồm Azure Pipelines, Agile Board, Test Plans và Azure Artifacts. Công cụ này hỗ trợ triển khai lên nhiều nền tảng cloud lớn, đặc biệt phù hợp với các dự án sử dụng hệ sinh thái Microsoft hoặc Azure.

1.5.6 GitLab CI

GitLab CI được tích hợp sẵn trong GitLab nên không cần phụ thuộc nhiều vào công cụ bên thứ ba. GitLab CI hỗ trợ template CI/CD để tự nhận diện ngôn ngữ lập trình, từ đó hỗ trợ build và test tự động, phù hợp với nhóm muốn quản lý mã nguồn, issue, merge request và pipeline trong cùng một nền tảng.

1.5.7 Travis CI

Travis CI hỗ trợ tự động build, test, quản lý deployment và gửi thông báo thông qua các build stage. Công cụ này phù hợp với các dự án cần pipeline đơn giản, dễ cấu hình và muốn tự động hóa các bước kiểm thử, triển khai sau mỗi thay đổi mã nguồn.

1.5.8 AWS CodePipeline

AWS CodePipeline là dịch vụ continuous delivery được quản lý hoàn toàn bởi AWS. Công cụ này tự động hóa quy trình build, test và deploy mỗi khi có thay đổi mã nguồn, phù hợp với các hệ thống triển khai trên AWS và cần tích hợp sâu với các dịch vụ như CodeBuild, CodeDeploy hoặc S3.

1.5.9 Bitbucket Pipelines

Bitbucket Pipelines tự động build, test và deploy theo mỗi commit, push hoặc pull request. Vì được tích hợp sẵn trong Bitbucket và kết nối tốt với Jira, Trello cùng các sản phẩm Atlassian, công cụ này phù hợp với nhóm đang sử dụng hệ sinh thái Atlassian để quản lý dự án và mã nguồn.

Most Popular CI / CD Tools Comparison

CI Tool	Hosting	Free version	Pricing	Supported platforms	Easy of use	Integrations
Jenkins	Self-hosted	Yes	Free. It is open-source	Linux, Windows, and macOS	Medium	GitHub, Bitbucket, GitLab, Jira, Slack
GitHub Actions	Cloud	Yes	Free execution units per tier, extra \$0.008/min	Linux, Windows, and macOS	High	GitHub native, Slack, external CI services
CircleCI	Cloud and self-hosted	Yes	Tiered; includes build minutes, pricing for extra minutes	Linux, macOS, Windows, GPU, ARM, and Docker	Medium	GitHub, Bitbucket, AWS, Slack
Azure DevOps	Cloud and self-hosted	Yes	Cloud-hosted: \$40/month, Self-hosted: \$15/month for extra pipelines	Linux, Windows, and macOS	Medium	Azure native, GitHub, Docker, Slack
GitLab CI	Cloud and self-hosted	Yes	Tiered pricing; \$10 for 1,000 additional build minutes	Linux, Windows, macOS, and Docker	Medium	GitLab native, GitHub, JIRA, Slack
Travis CI	Cloud and self-hosted	No	Starting at \$64/month with 1 concurrent job, unlimited build minutes	Linux, macOS, and iOS	High	GitHub, Bitbucket, Docker
AWS CodePipeline	Cloud	Yes	Pay per pipeline, extra costs for storage	Linux, Windows, and macOS	Medium	AWS native, GitHub, Bitbucket, Docker
Bitbucket Pipelines	Cloud	Yes	Free tier includes build minutes, extra \$10/month for 1,000 additional minutes	Linux, Windows, and macOS	High	Bitbucket native, Slack, JIRA
TeamCity	Cloud and self-hosted	Yes	Cloud starts at \$45/month; on-premises includes 3 free build agents with unlimited build time	Linux, Windows, macOS, and Docker	Medium	GitHub, GitLab, Docker, Slack



Hình 3: Một số công cụ CI/CD phổ biến

1.6 Các yếu tố so sánh và lựa chọn công cụ CI/CD

Khi chọn nền tảng CI/CD, nhóm phát triển nên cân nhắc các yếu tố sau:

- **So sánh giữa giải pháp dựa trên đám mây (Cloud-based) và giải pháp tự lưu trữ (Self-hosted):** Nền tảng cloud-based thường có sẵn giao diện quản lý pipeline và không cần tự cài đặt hoặc bảo trì hạ tầng. Ngược lại, self-hosted cho phép nhóm tự quyết định nơi đặt build server hoặc agent, phù hợp khi cần kiểm soát cao về bảo mật, tài nguyên và môi trường thực thi.
- **Tính dễ dùng:** Công cụ nên có giao diện thân thiện, tài liệu rõ ràng và cách cấu hình dễ hiểu để nhóm có thể nhanh chóng xây dựng, theo dõi và sửa lỗi pipeline.
- **Khả năng tích hợp:** Nền tảng CI/CD cần tích hợp tốt với ngôn ngữ lập trình, framework, công cụ quản lý mã nguồn, công cụ theo dõi lỗi, hệ thống kiểm thử và nền tảng cloud mà dự án đang sử dụng.
- **Khả năng cấu hình:** Công cụ nên cho phép cấu hình linh hoạt từ yếu tố kích hoạt khởi chạy pipeline, thứ tự các bước build/test/deploy, biến môi trường, các thông tin nhạy cảm cho đến cách xử lý khi build hoặc test thất bại. Việc cấu hình có thể thông qua script, file YAML hoặc giao diện UI.
- **Chi phí phù hợp với ngân sách tổ chức:** Cần cân nhắc chi phí sử dụng nền tảng, số phút build, số lượng người dùng, tài nguyên máy chạy pipeline, chi phí lưu trữ artifact và chi phí vận hành nếu tự lưu trữ. Một công cụ phù hợp không chỉ đáp ứng yêu cầu kỹ thuật mà còn phải nằm trong ngân sách dài hạn của tổ chức.
- **Mức độ hiểu biết của đội ngũ:** Cần xem xét kỹ sư trong nhóm đã có kinh nghiệm với nền tảng đó hay chưa, cộng đồng hỗ trợ có lớn không và tài liệu hướng dẫn có đầy đủ không. Một công cụ mạnh nhưng đội ngũ chưa quen có thể làm tăng chi phí học tập và bảo trì.

2 Test Harness Engineering

2.1 Khái niệm Test Harness

Test harness là tập hợp phần mềm, dữ liệu đầu vào và các thành phần hỗ trợ được tạo ra để kiểm tra một phần mềm khi chạy trong nhiều môi trường khác nhau, sau đó phân tích kết quả nhằm hỗ trợ integration testing.

Test harness là tập hợp các stub, driver và công cụ hỗ trợ automation testing cũng như debugging. Nó cung cấp bản sao của các tài nguyên hoặc cấu hình chưa có sẵn trong môi trường test. Ví dụ, nếu phần mềm cần tích hợp với một hệ thống PHP nhưng trong giai đoạn phát triển chưa có bản triển khai PHP thực tế, test harness có thể được tạo ra để thay thế hệ thống đó.

2.2 Cấu tạo của Test Harness

Test harness gồm hai phần chính là **test execution engine** và **test script repository**. Test execution engine được dùng để kiểm tra phần mềm, còn test script repository là nơi lưu trữ test case và script kiểm thử.

Một test harness chứa toàn bộ thông tin cần thiết để biên dịch và thực thi một bài test. Vì vậy, nó có thể được sử dụng cho cả integration testing lẫn automation testing, đặc biệt trong những trường hợp cần mô phỏng môi trường hoặc thành phần chưa hoàn chỉnh.

2.3 Đặc điểm của Test Harness

- Hỗ trợ automation testing bằng cách gọi các hàm và so sánh kết quả thực tế với kết quả kỳ vọng.
- Là nơi chứa code đã phát triển và có thể được kiểm chứng bởi một automation framework.
- Có thể trở thành một phần của deliverable code trong hệ thống phần mềm.
- Có thể sao chép lại hành vi của phần mềm thật để phục vụ kiểm thử.
- Hỗ trợ thực hiện các thao tác trên phần mềm đang được kiểm thử.
- Có thể tích hợp vào các composite framework để mở rộng khả năng kiểm thử.

2.4 Mục đích sử dụng

- Thúc đẩy automation testing, từ đó tăng năng suất sử dụng tài nguyên và phần mềm.
- Chạy toàn bộ test suite và tạo báo cáo test sau khi thực thi.
- Đo lường code coverage để đánh giá mức độ bao phủ của kiểm thử.
- Xử lý các tình huống thực tế khó lặp lại trong môi trường kiểm thử thông thường.
- Chạy test trong một test suite bằng framework, đồng thời hỗ trợ debug và theo dõi kết quả.
- Nâng cao chất lượng phần mềm thông qua việc kiểm thử có hệ thống và có thể lặp lại.

2.5 Quy trình xây dựng Test Harness

1. Tester và developer viết test script cho từng tình huống, input và dữ liệu cụ thể. Quá trình này thường bắt đầu bằng việc tạo dữ liệu đúng, dữ liệu nhạy cảm và dữ liệu đại diện cho các trường hợp quan trọng.
2. Sau khi kích hoạt test harness, tester có thể chạy toàn bộ test và tạo kết quả chỉ bằng một lệnh hoặc một thao tác tự động.
3. Hệ thống thu thập kết quả thực thi từ các test case đã chạy.
4. Test harness so sánh kết quả kỳ vọng với kết quả thực tế để tìm ra sai lệch, sau đó ghi nhận lỗi hoặc bất thường.
5. Báo cáo test được tạo ra và chia sẻ với các bên liên quan để tiếp tục phân tích, sửa lỗi hoặc cải thiện chất lượng phần mềm.

2.6 Các tình huống áp dụng

Automation Testing: Test harness có thể được dùng làm nền tảng để chạy test tự động hiệu quả hơn. Sau khi thực thi, hệ thống tạo báo cáo chi tiết để tester và developer theo dõi lỗi, đánh giá kết quả và cải thiện chất lượng mã nguồn.

Integration Testing: Khi các thành phần riêng lẻ chưa được phát triển đầy đủ, driver và stub trong test harness có thể được dùng để thay thế các thành phần chưa sẵn sàng. Nhờ đó, nhóm phát triển vẫn có thể kiểm tra luồng tích hợp mà không phải chờ toàn bộ hệ thống hoàn thiện.

2.7 Công cụ hỗ trợ Test Harness

- **JUnit:** Đây là unit testing framework cho Java, đóng vai trò quan trọng trong test-driven development và thuộc họ công cụ xUnit.
- **NUnit:** Đây là unit testing framework cho nền tảng .NET, thường được dùng để viết và chạy các bài kiểm thử tự động cho ứng dụng .NET.

2.8 Ưu điểm và nhược điểm của Test Harness

Ưu điểm:

- Cải thiện năng suất của tester và developer trong quá trình kiểm thử.
- Hỗ trợ test automation, giúp quá trình kiểm thử có thể lặp lại và ít phụ thuộc vào thao tác thủ công.
- Hỗ trợ debug code nhờ khả năng ghi nhận, so sánh và phân tích kết quả kiểm thử.
- Phát hiện lỗi sớm trong vòng đời phát triển phần mềm SDLC.
- Đo lường code coverage để đánh giá mức độ kiểm thử của mã nguồn.
- Bao phủ được các use case phức tạp hoặc khó tái hiện thủ công.
- Tăng tính ổn định và chất lượng phần mềm nhờ kiểm thử kỹ ở cấp độ unit và integration.

Nhược điểm:

- Không hỗ trợ record và playback như một số công cụ kiểm thử giao diện chuyên biệt.
- Đòi hỏi kỹ năng kỹ thuật để thiết kế, triển khai và duy trì test harness.
- Tốn chi phí và thời gian để xây dựng automated test suite, đặc biệt khi cần nhân sự có kỹ năng cao.

Tài liệu tham khảo

1. TechWorld with Milan, *What is CI/CD Pipeline*. Dùng tham khảo cho phần CI/CD và các yếu tố lựa chọn công cụ.
<https://newsletter.techworld-with-milan.com/p/what-is-cicd-pipeline>
2. ITviec Blog, *CI/CD là gì?*. Dùng tham khảo cho phần khái niệm và tổng quan CI/CD.
<https://itviec.com/blog/ci-cd-la-gi/>
3. Tutorialspoint, *Software Testing - Test Harness*. Dùng tham khảo cho phần Test Harness Engineering.
https://www.tutorialspoint.com/software_testing_dictionary/harness.htm